

Natural Language Processing (NLP)

Large Language Models

LLMS

By:

Dr. Zohair Ahmed



www.youtube.com/@ZohairAI

Subscribe



www.begindiscovery.com

Semi-Supervised vs Self-Supervised vs Supervised

- **Self-Supervised Learning (pretraining)**
 - No labels
 - Model creates labels from the data (e.g., guess next word)
 - Used to learn **general language understanding**
- **Supervised Learning (fine-tuning)**
 - Labels provided by humans
 - Used for a specific task
 - Usually small dataset
- **Semi-Supervised Learning**
 - Mix of:
 - a. a few labeled examples
 - b. lots of unlabeled examples
 - Used when labels are expensive or rare
 - The model benefits from both types of data

One Transformer, Many NLP Tasks

- **Sentiment analysis:** text → class label.
- **Summarization:** long text → short text (same language).
- **Question answering** context + question → answer span/tokens.
- **Unifying view:** Same model backbone; swap objective or small task head.
- **Example**
 - **Input:** “The product broke in two days.” → **Output:** “negative.”
 - **Input:** 3-paragraph article → **Output:** 2-sentence summary.
 - **Input:** Paragraph + “What year was X founded?” → **Output:** “2004.”



The Bottleneck: Labeled Data Requirements

- Large Transformers have many parameters; training from scratch is data-hungry.
- Many tasks/languages lack big, high-quality labeled datasets.
- Cold-starting each task from random parameters is slow and inefficient.
- **Example**
- **Low-resource language sentiment:** maybe thousands of examples available vs. millions typically needed for training from scratch.
- How Many Data for Urdu?



Why Labeling Is Expensive

- Requires skilled annotators; quality control and guidelines.
- Complex tasks (QA, summarization) are time-consuming to label.
- Domain-specific or multilingual labeling increases cost further.
- **Example**
- **QA labeling:** write question, read context, select/compose answer → minutes per item at scale.



The Opportunity: Unlabeled Text Is Abundant

- Massive raw text exists: Wikipedia, news, blogs, books, forums.
- “Unlabeled” = text without task tags (no class/summary/answers).
- **Question:** Can raw text alone teach a model usable language skills?
- **Example**
- A corpus of articles about science, sports, politics no labels, just sentences and paragraphs.



Two Key Questions

- **Objective:** What exactly do we train on if there are no labels?
- **Utility:** Will this pretraining truly reduce the labeled data needed later?
- **Example**
- The model should learn syntax, semantics, and world-knowledge patterns just by reading text.



How We Learn Language

- Assume that we have a questions to a lay person
- “Wow Pakistan has now reached the Moon”
 - Is the sentence expressing a positive sentiment or negative sentiment.
- He likes to stay, he likes to stray, he likes to sway,
 - Are these meaningful sentences.
- You don't train for these tasks.
- You just Expose large amount of Knowledge.



How We Learn Language

- People do sentiment, basic QA, and grammaticality judgments without explicit per-task training.
- We absorb patterns by exposure (reading, listening, conversation, media).
- Hypothesis: Similar exposure can shape a model's internal representations.
- **Example**
- Hearing praise words in positive contexts → we learn “praise = positive” implicitly.

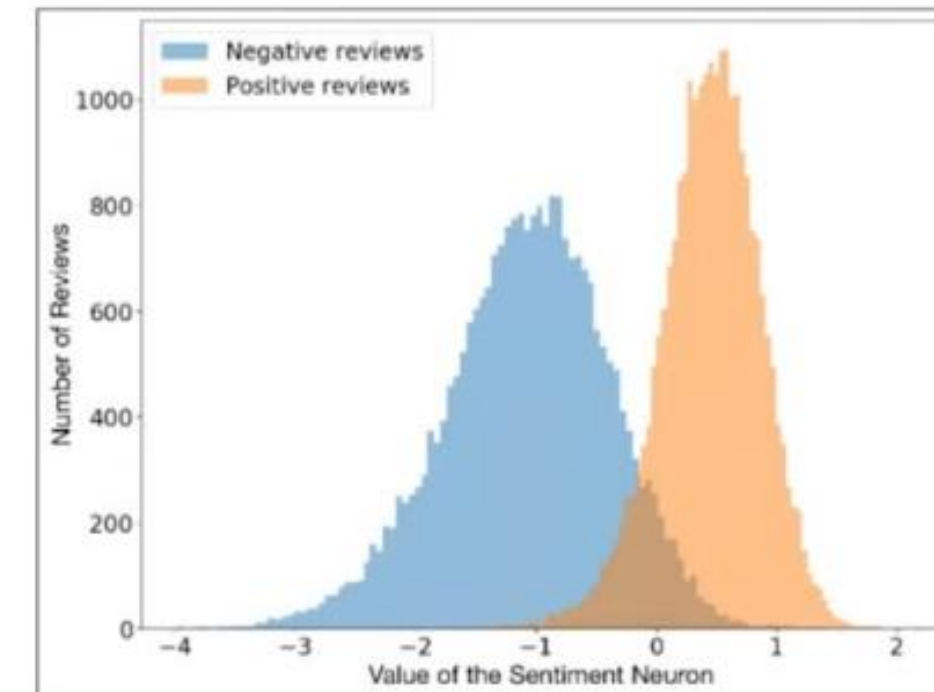
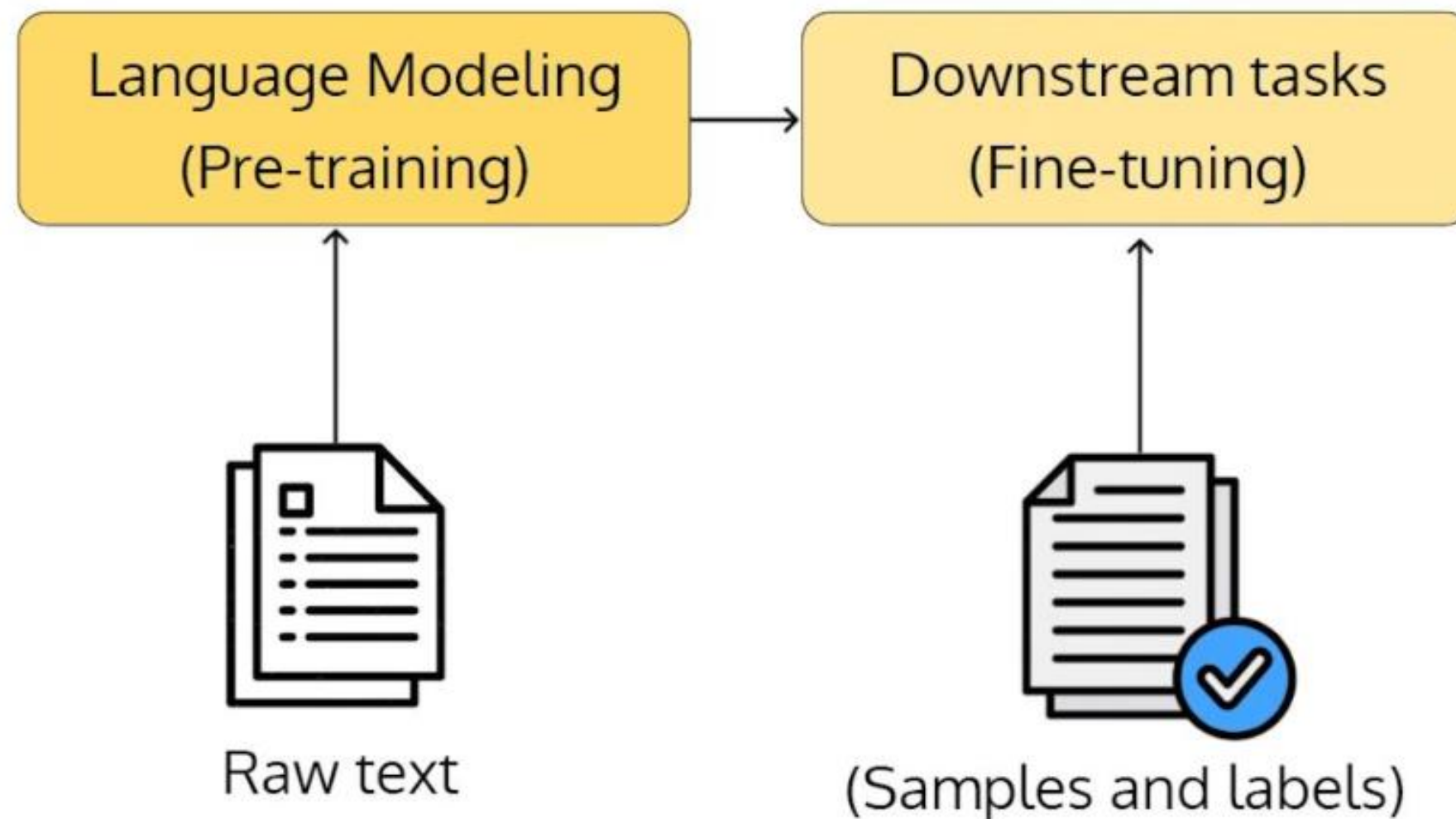


Idea

- Can a model develop basic understanding of language by getting exposure to a large amount of raw text? → **Pre Training.**
- More importantly, after getting exposed to such raw data can it learn to perform well on downstream tasks with minimum supervision? → **Supervised Fine-tuning.**



Idea



With this representation a linear model classifies reviews with 91.8% accuracy beating the SOTA (Ref)

...matches the performance of previous supervised systems using 30-100x fewer labeled examples (Ref)

Pretraining

- **Building General Language Understanding**
- Train on raw text with a self-supervised objective (no human labels).
- The model adopts patterns: word order, agreement, collocations, discourse cues.
- Output: A “language-aware” backbone you can adapt to many tasks.
- **Example**
- After pretraining, the model recognizes that “cat” and “cats” are related, or that “because” introduces reasons.



Fine-tuning

- **Specializing With Few Labels**
- Start from the pretrained model; add a small labeled dataset for your task.
- Adjust (fine-tune) the parameters to optimize the task objective.
- **Benefit:** Far fewer labels needed than training from scratch.
- **Example**
- With 5k labeled sentiment reviews, fine-tuning yields strong accuracy vs. needing 100k+ from scratch.



Evidence

- **Evidence: It Works in Practice**
- Point-in-time results (example: sentiment): **~92%** accuracy after pretraining + fine-tuning.
- Matched/beat prior supervised-only systems with 30× 100× fewer labels.
- Pretraining meaningfully reduces supervision requirements.
- **Example**
- Same backbone reused for QA and summarization with task-specific fine-tuning, outperforming scratch baselines.



Why This Matters (Tasks & Languages)

- Low-resource settings benefit most (limited labels).
- Faster development: one backbone, many tasks.
- Consistent engineering: reuse data processing, training pipelines.
- **Example**
- Transfer from English-heavy pretraining to Urdu sentiment with modest labeled Urdu data.



What is Language Modeling?



What is Language Modeling?

- The modern AI strategy involves two main phases: **Pre-training** and **Fine-tuning**.
- **What is Pre-training?**
- It's like sending a model to a "language university." Before it can become a specialist (like a translator or a question-answerer), it first needs to gain a general, comprehensive *understanding of language* itself.
- This initial, broad learning phase is called pre-training.
- **How Do We Achieve This (The Formal Task)?**
- We can't just tell a machine to "understand language." We need a concrete, measurable task.
- That task is **Language Modeling (LM)**.
- **Pre-training is, in practice, the process of training a large-scale Language Model.**
- Emphasize the distinction between the *goal* (understanding language) and the *method* (language modeling).
The modern AI was realizing that successfully training a model on the method is a powerful way to achieve the goal.



The BIG Idea: Connecting "Understanding" to "Predicting"

- **The Central Question:** How do we practically teach a machine to "understand language"?
- **The Simplified, Powerful Idea:** We train it to master the task of **predicting the next token in a sequence.**
- **The Core Argument:**
 - To consistently and accurately predict the next word, a model *must* develop a sophisticated understanding of the preceding context.
 - To know that **"book"** is a better prediction than **"thermometer"** after **"I enjoyed reading a..."**, the model needs to understand concepts related to enjoyment, reading, and objects that can be read.
- Therefore, if a model gets very good at prediction, it has by necessity learned about grammar, syntax, facts, and even reasoning patterns embedded in the language.
- **Acknowledging the Nuance:**
 - There is an active debate on whether these models *truly* "understand" or are just incredibly sophisticated pattern-matching systems ("stochastic parrots").
 - For our purposes, we will proceed with the argument that **mastery of prediction is a powerful proxy for understanding.**



The Data Goldmine: Creating Training Data for Free

- **The Breakthrough: Self-Supervised Learning**
- If our task is "predict the next word," the internet becomes a near-infinite source of free training data. We don't need humans to manually label anything. The data labels itself!
- **How to Create Countless Training Samples from One Sentence:**
 - **Source Sentence:** "FAST NUCES is a top computing institute."
 - **Sample 1:**
 - a. Input (X): ["FAST"]
 - b. Target Label (Y): "NUCES"
 - **Sample 2:**
 - a. Input (X): ["FAST", "NUCES"]
 - b. Target Label (Y): "is"
 - **Sample 3:**
 - a. Input (X): ["FAST", "NUCES", "is"]
 - b. Target Label (Y): "a"
 - ...and so on for the entire sentence.
- **The Power of Scale:**
 - A single sentence of 15 words can generate 14 distinct training examples.
 - Now, scale this up to the entire web: trillions of sentences, leading to quadrillions of training examples.
 - This is what allows us to train the massive "Large" Language Models. The labels (the next word) are **free** because a human already wrote the full sentence.



How the Model Learns: Predicting Probability Distributions

- **It's Not About One Word; It's About All of Them.**
- When we give the model an input (e.g., ["I am going to the"]), we are **not** asking it to output a single word.
- **The Actual Task:** The model must output a **probability distribution** over its *entire vocabulary*.
 - This is a list of tens of thousands of probabilities, one for every possible word/token it knows.
- **Example Output Distribution for the input ["I am going to the"]:**
 - $P(\text{"gym"} \mid \text{context}) \rightarrow 0.35$ (35%)
 - $P(\text{"store"} \mid \text{context}) \rightarrow 0.22$ (22%)
 - $P(\text{"movies"} \mid \text{context}) \rightarrow 0.15$ (15%)
 - ...
- $P(\text{"moon"} \mid \text{context}) \rightarrow 0.0001\%$
- $P(\text{"equation"} \mid \text{context}) \rightarrow 0.00005\%$
- **The Learning Process (Loss Function):**
 - The model makes a prediction (the distribution above).
 - We check the probability it assigned to the *actual* next word from the text (e.g., if the real sentence was "I am going to the gym," we look at the 35%).
 - The model is **penalized** based on how "surprised" it was (i.e., how low the probability was for the correct word).
 - It then adjusts its internal parameters (billions of them) to assign a higher probability to the correct word next time it sees a similar context.
 - Repeat this process billions or trillions of times.



Example: Penalized Based

- Sentence in training: **"The doctor checked the patient's blood pressure."**
- At the word **"pressure"**:
- $\text{Penalty} = -\log(P(\text{correct word}))$ (We use log because tiny probabilities like 0.0001 become big penalties)
- Model thinks:
- **"After 'blood', next is temperature!"** → 99% sure
Real word: "pressure" → only 1% chance
- **Model:** 🤪 "I was SO wrong!" →
- **Teacher:** "Bad! Big penalty!"
- **Model remembers:** "blood" + "pressure" = common!

What the model predicted	Chance, it gave "pressure"	Surprise?	Penalty
Thought "temperature" was likely	$P(\text{pressure}) = 0.01 \text{ (1\%)}$	😱 VERY surprised	BIG penalty
Already knew "blood pressure" is common	$P(\text{pressure}) = 0.8 \text{ (80\%)}$	😊 Not surprised	Tiny penalty

P(correct)	$-\log(P) \rightarrow$ Penalty
1.0 (100%)	0 → No penalty
0.5 (50%)	~0.7 → small
0.01 (1%)	~4.6 → big
0.0001	~9.2 → HUGE

Tying It All Together: The Grand Vision of Pre-training

- Let's connect all the dots.
- **The Goal (Intuitive):** Create a model that understands human language, so it can be adapted to many useful tasks (e.g., summarization, translation, Q&A). This is **pre-training**.
- **The Method (Formal):** Frame this goal as a concrete task: **Language Modeling** predicting the next word in a sequence based on the context.
- **The Fuel (Data):** Use **self-supervised learning** to turn the massive, unstructured text of the internet into a virtually infinite supply of (context, next_word) training examples.
- **The Engine (Model):** Use a powerful architecture like the Transformer to learn the complex patterns from this data. The model learns by predicting a probability distribution over the vocabulary and getting corrected billions of times.
- **The Outcome:** After this massive pre-training phase, the model's internal parameters now encode a deep and nuanced statistical representation of language grammar, facts, reasoning styles, and contextual relationships. It has become a powerful, general-purpose **Foundation Model**.



Tying It All Together: The Grand Vision of Pre-training

Text corpus



(Self-supervised)
Training

Pretrained LM



Adaptation

Tasks

Question
Answering



Text
Classification



Information
Retrieval



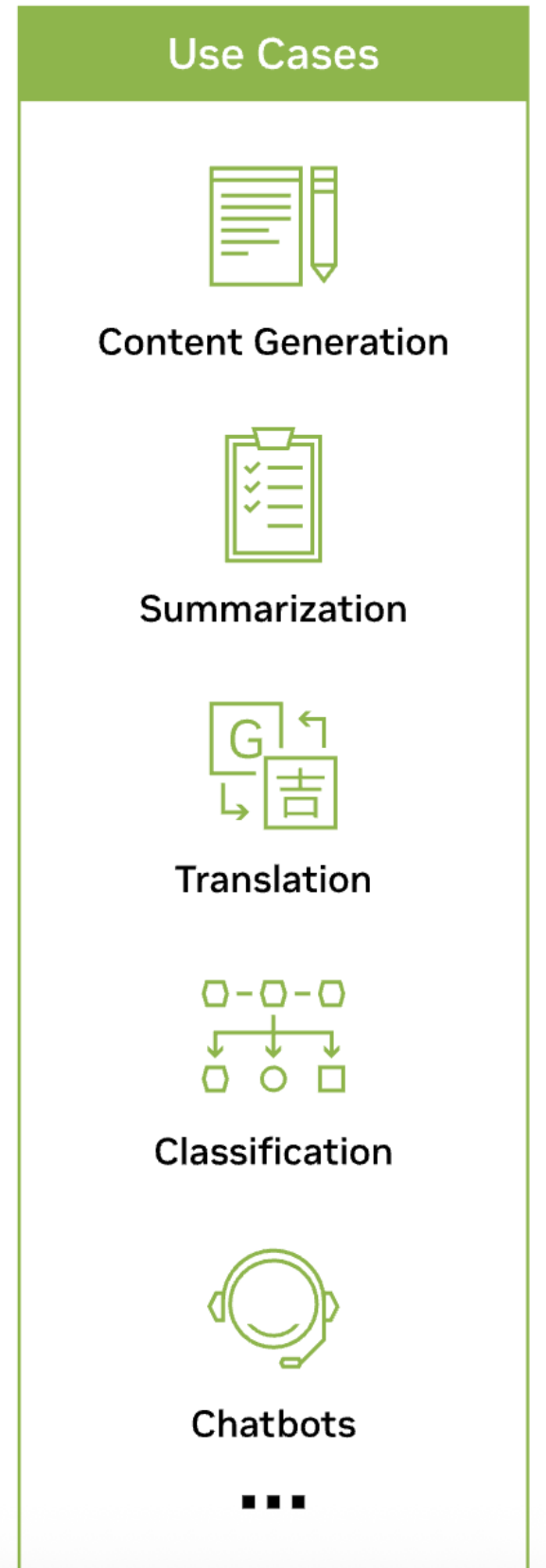
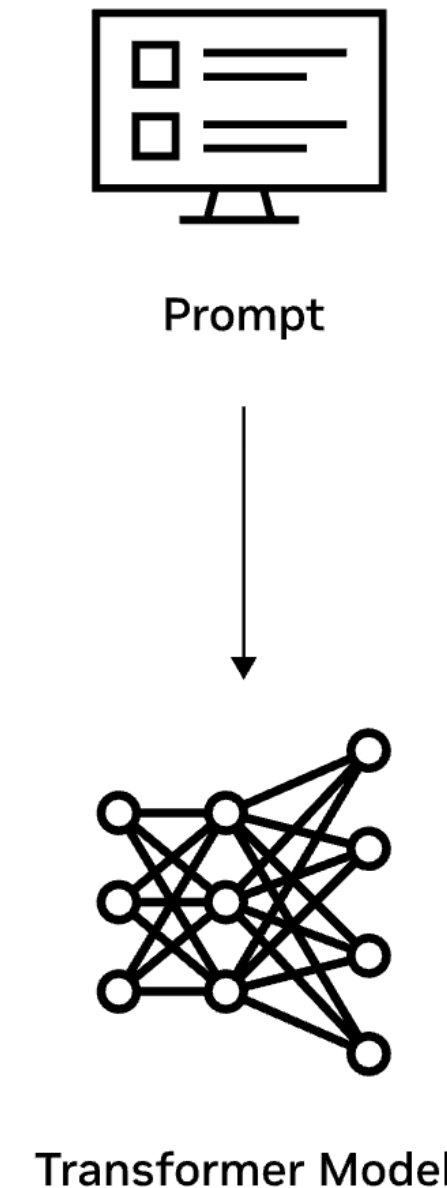
⋮

Large
Language
Models

Introduction to Large Language Models

Large Language Models

- Large Language Models (LLMs) are **Computational agents that can understand and generate human language.**
- They can have conversations, answer questions, summarize text, translate languages, and much more.
- These models have **transformed the fields of NLP and AI** because they can perform many tasks without being explicitly programmed for each one.



N-gram Language Model (from classical NLP)

- A **bigram model** looks at **1 previous word** – love, coding, 5
to predict the **next word**.
- Example sentence: I love NLP
- The **bigrams** in this sentence are:
 - I → love
 - love → NLP
- So the model learns patterns like:
 - **Previous word, Next word, Count**
 - love, NLP, 10
 - love, pizza, 3



N-gram Language Model (from classical NLP)

- **Example**
- I love NLP
 - [I, love]
 - [love, NLP]
- **Step 1: Tokenize the text**
 - we **do** need to make tokens first (words, or subwords).
- ["I", "love", "NLP"]
- **Step 2: Count how often each bigram appears**
- From a large corpus, you count patterns like:
 - after "I" → what words usually come?
 - after "love" → what words usually come?
- **Step 3: Convert counts to probabilities**
- Suppose the word **love** is followed by these words in the training data:
 - **Next word, Count, Probability**
 - NLP, 50, $50/100 = 0.50$
 - you, 30, $30/100 = 0.30$
 - pizza, 20, $20/100 = 0.20$
- So the model learns:
- $P(\text{next word} = \text{NLP} \mid \text{previous word} = \text{love}) = 0.50$
- **Step 4: Predict the next word**
- If your previous word is **love**, the bigram model predicts:
 - NLP (most probable)
- or
- samples randomly based on these probabilities



N-gram Language Model (from classical NLP)

- **N-gram models**
- Tokenize text
- Count n-grams
- Compute probabilities
- Use probabilities to predict the next word
- **Training:**
 - Text → Tokenize → Count bigrams → Compute $P(\text{next word} \mid \text{previous word})$
- **Prediction:**
 - Previous word → Look up probabilities → Choose next word



Connection to the n-gram Language Model (from classical NLP)

- **What the n-gram model does**
- You may remember the simple n-gram model from earlier NLP:
 - It assigns a **probability** to a sequence of words.
 - It predicts text by **choosing the next word based on previous words**.
 - It learns by **counting how often word combinations occur** in a large text dataset.
- **How LLMs are similar**
- LLMs also:
 - Assign probabilities to word sequences.
 - Generate text by predicting the next word.
- **How LLMs are different**
- Instead of using simple counts:
- LLMs are trained to **guess the next word using neural networks**.
- They learn **deep patterns**, meaning:
 - grammar
 - facts about the world
 - writing style
 - reasoning patterns
 - relationships between concepts
- So the basic idea (predicting the next word) is the same, but the **method and scale** are vastly more powerful.



Fundamental Intuition

- Why do LLMs work so well?
- The key idea is:
- **“Text contains enormous amounts of knowledge.”**
- When an LLM reads billions or trillions of words from books, articles, websites, etc., it naturally absorbs:
 - language rules
 - world knowledge
 - facts
 - patterns in how humans express ideas
 - reasoning structures
 - relationships between concepts
- This process is called **pretraining**.
- Pretraining is what gives a language model its broad abilities even before it's fine-tuned for specific tasks.



What does a model learn from pretraining?

- **1. Vocabulary and contextual meaning**

- “With roses, dahlias, and peonies, I was surrounded by **flowers**.”

The model learns:

- these are names of flowers
- the category (“flowers”)
- context and relationships

- **2. Understanding adjectives and nuance**

- “The room wasn’t just big it was **enormous**.”

It learns:

- “big” and “enormous” have related meanings
- intensification in language

- **3. Mathematical facts**

- “The square root of 4 is **2**”

- Not deep mathematics
- but memorized patterns and simple arithmetic frequently seen in text.

- **4. Factual/world knowledge**

- “The author of A Room of One’s Own is Virginia **Woolf**.”

- It learns factual associations like:

- titles of famous works
- the authors who wrote them

- **5. Common sentence structures**

- “The doctor told me that **he** ...”

It learns:

- how sentences normally continue
- typical narrative patterns
- common verb–object relationships

- Even though the model is only trained to predict the next word, it ends up learning **grammar, semantics, facts, reasoning patterns, and world knowledge.**

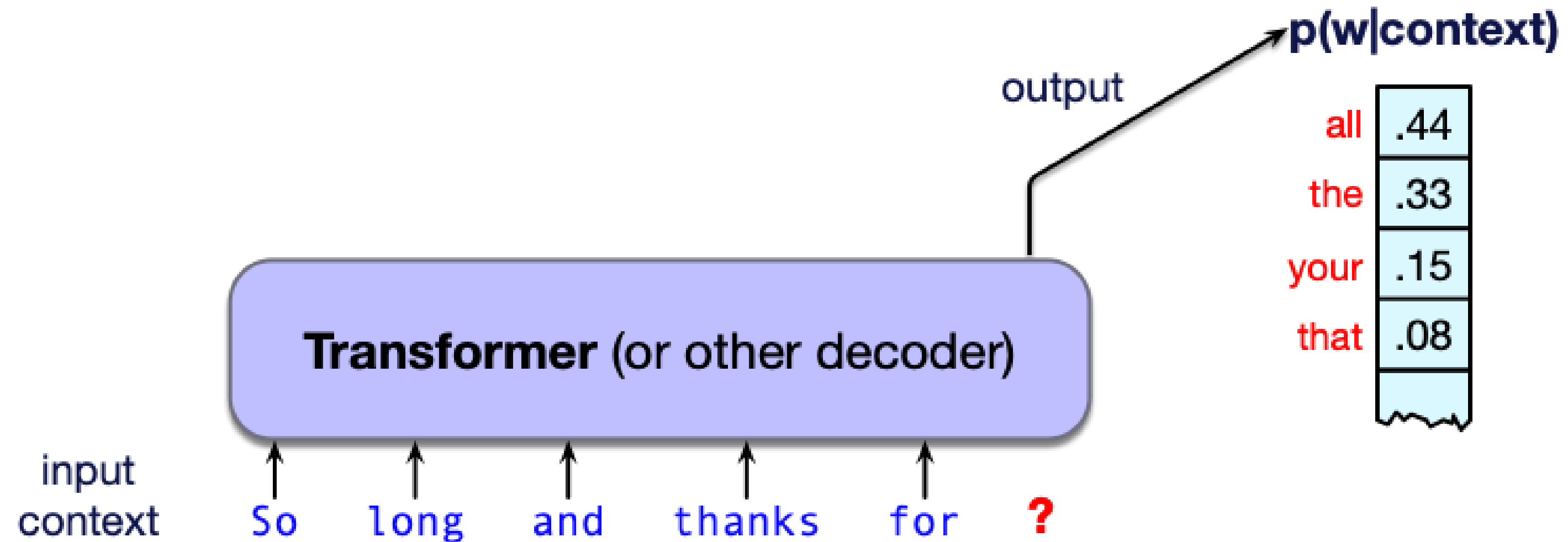


What is a large language model?

A neural network with:

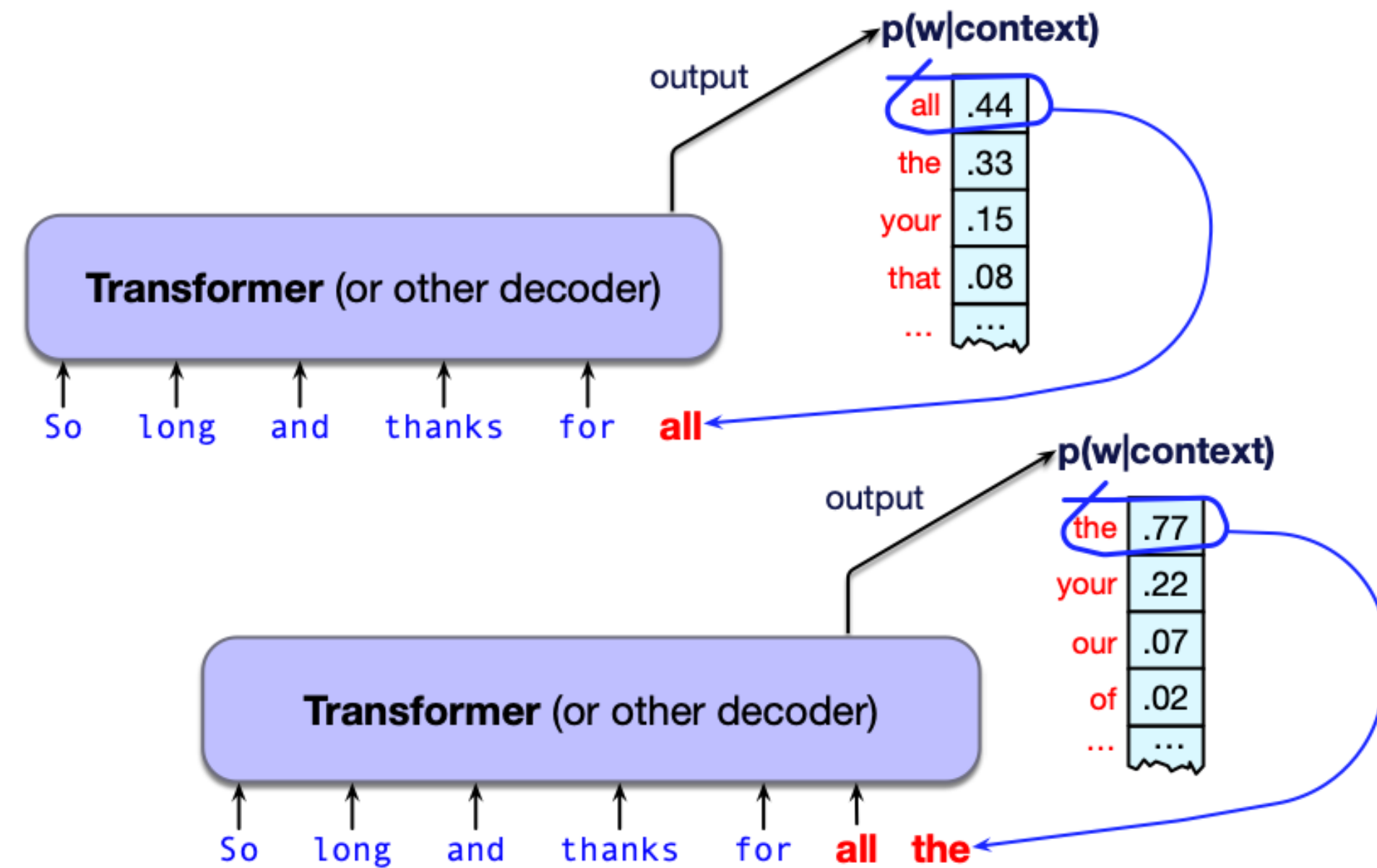
Input: a context or prefix,

Output: a distribution over possible next words

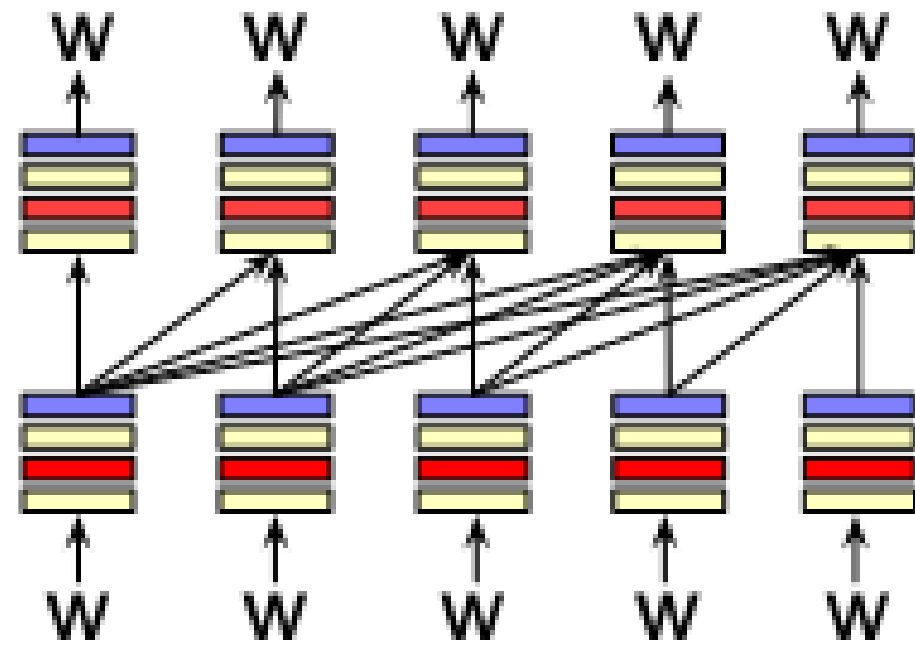


LLMs can generate!

- Transformer-based language model (or any decoder model) generates text **one word at a time**.
- At each step:
- The model looks at all the words so far → this is the **context**.
- It outputs a **probability distribution** over *all possible next words*.
- The model then **samples** (picks) the next word based on those probabilities.
- The chosen word is added to the context.
- Repeat the process to generate the next word.
- This cycle continues until the model finishes the sentence.



Three Architectures For Large Language Models

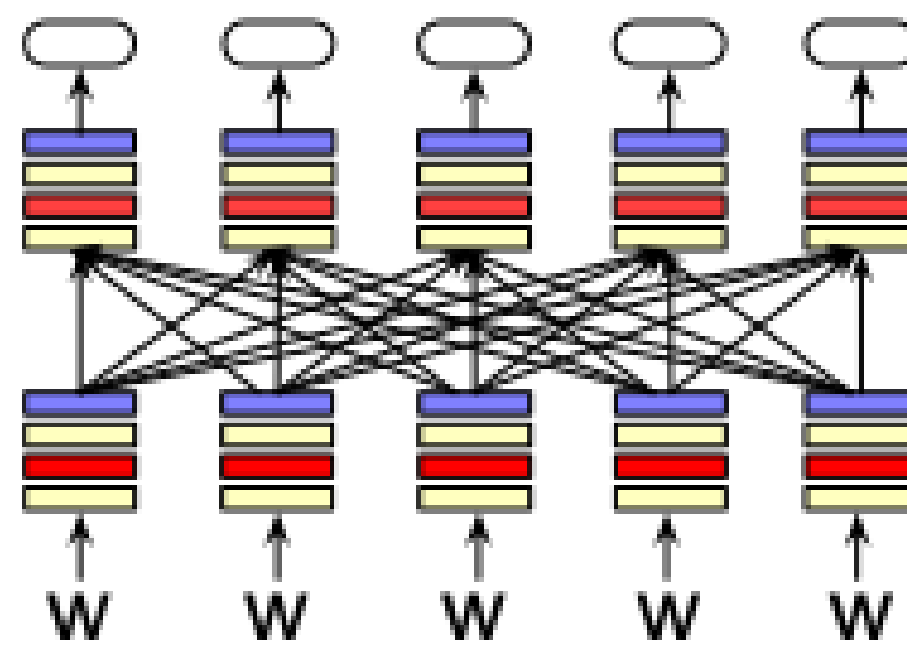


Decoders

GPT, Claude

Llama

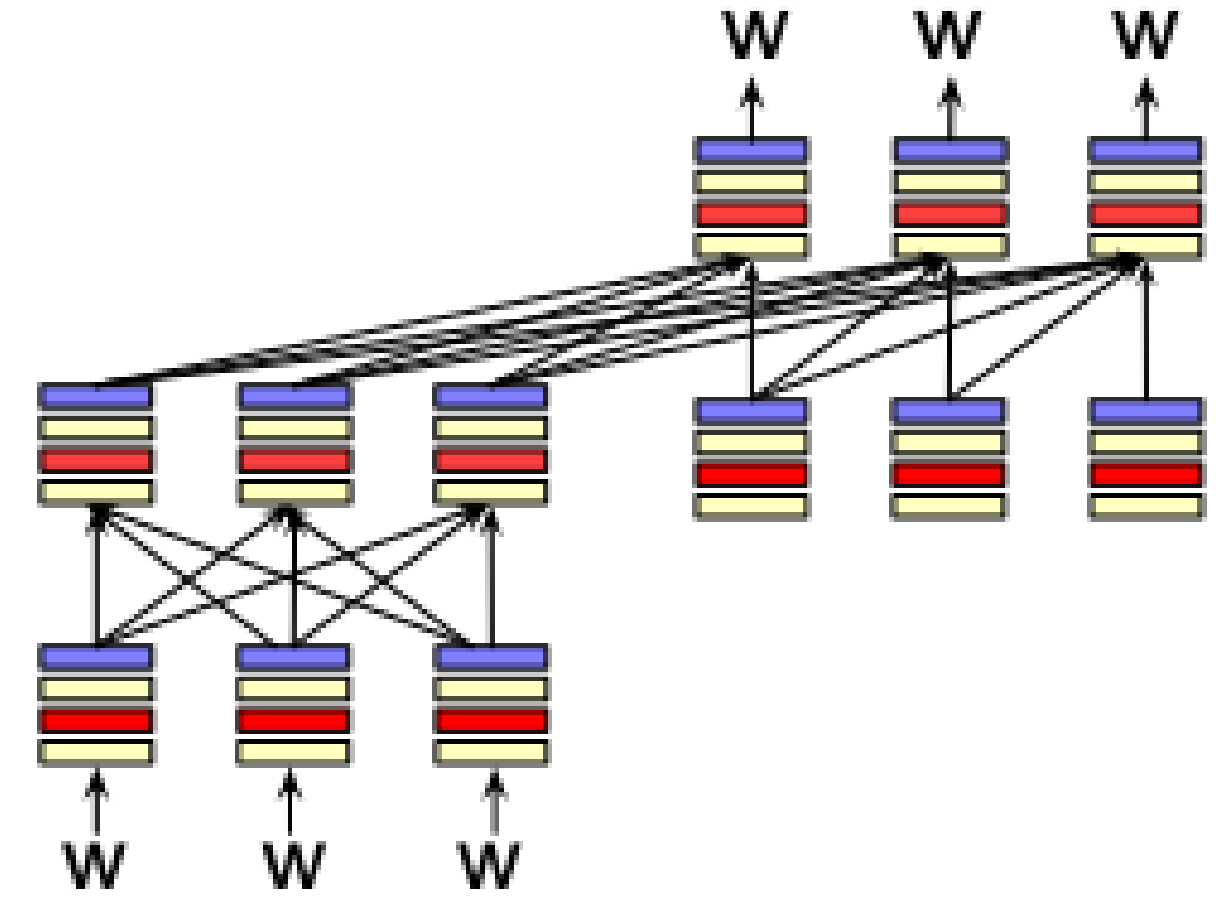
Mixtral



Encoders

BERT family

HuBERT



Encoder-decoders

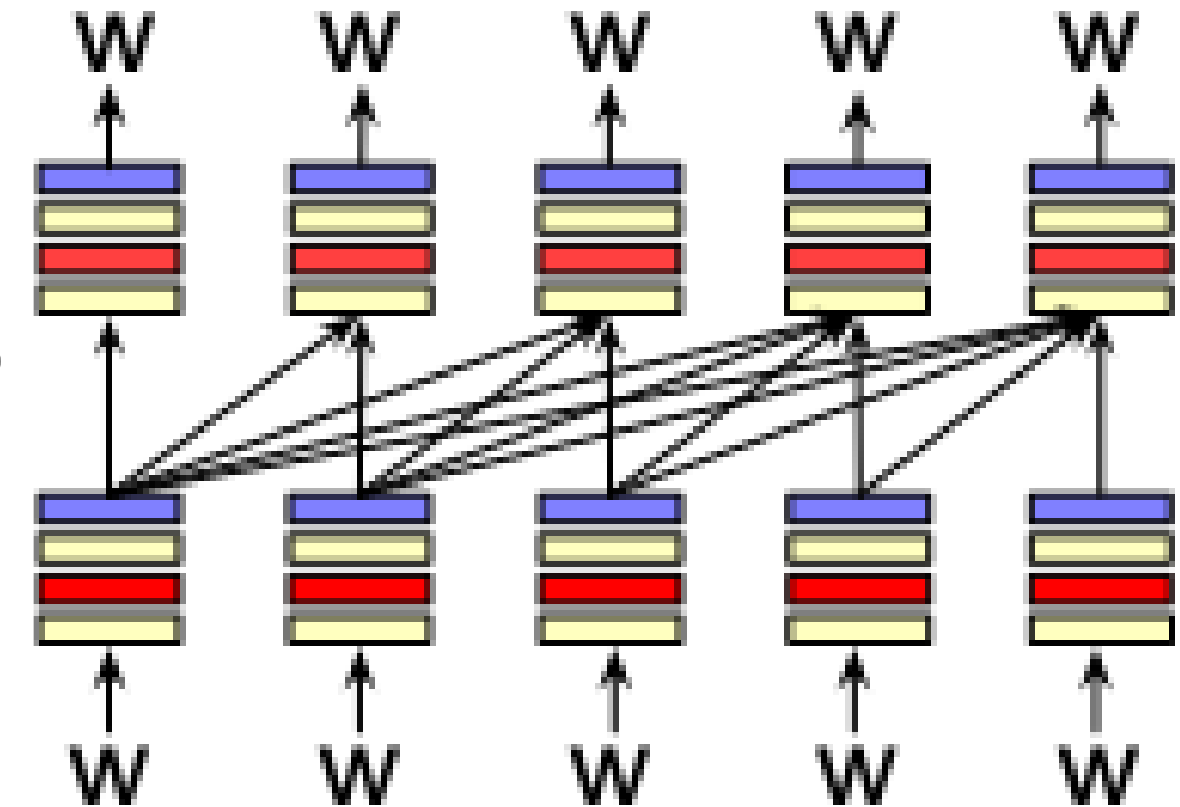
Flan-T5, Whisper

Decoder-Only Models (Autoregressive Models)

- These are the models that most people think of when they hear “LLM.”
- **Examples**
 - GPT (OpenAI)
 - Claude (Anthropic)
 - Llama (Meta)
 - DeepSeek
 - Mistral
- **How they work**
- They take a sequence of tokens (words/subwords) and then **generate the next token one at a**

time, moving left to right.

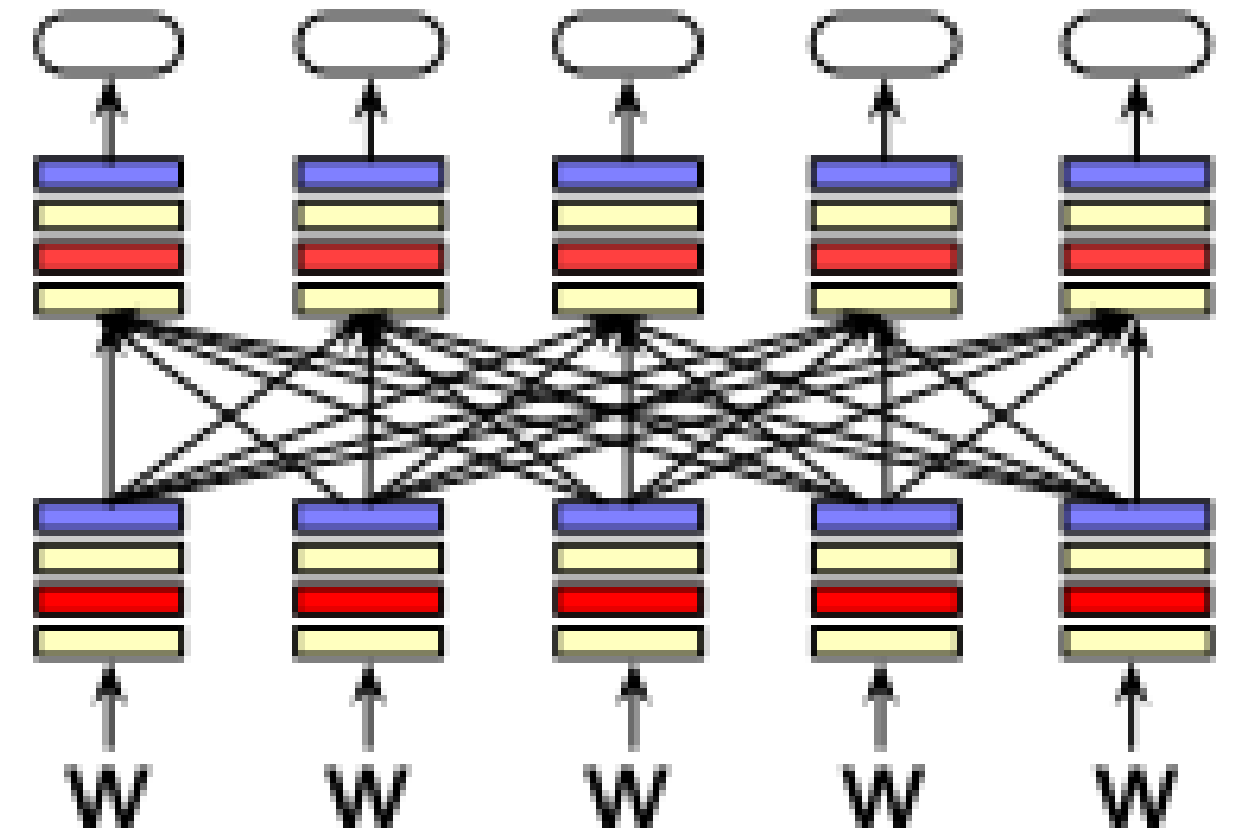
- This is called:
 - **Causal Language Modeling (CLM)**
 - **Autoregressive Generation**
- **They are generative**
- You give them a prompt
- They generate text step-by-step
- **Use cases:**
 - Chatbots
 - Story and code generation
 - Reasoning
 - Long text completion



Encoder-Only Models (Masked Language Models)

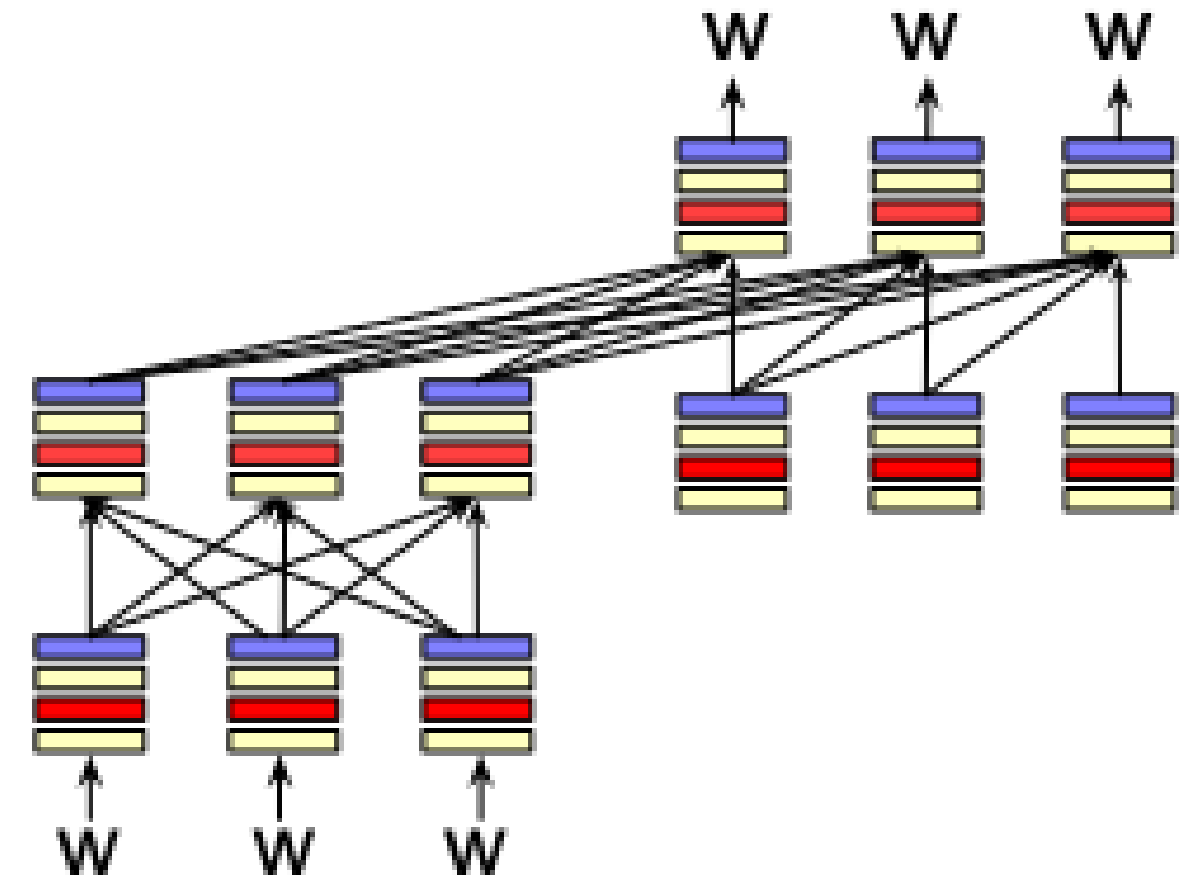
- These are **not** generative like GPT. Instead, they are excellent at **understanding** text.
- **Examples**
 - BERT
 - RoBERTa
 - DistilBERT
 - ALBERT
- **How they work**
- They use **masked language modeling (MLM)**:
- Some words are hidden (masked) in a sentence
- The model tries to guess the missing words based on **both left and right context**

- **Example:**
 - The doctor ____ me to rest for a week.
- The model predicts “told”.
- **Typically used for supervised tasks**
- They are usually **fine-tuned** on labeled data for tasks like:
 - Classification (sentiment analysis, topic classification)
 - Named Entity Recognition
 - Question answering
 - Document understanding
- **They don't generate long text. They analyze it.**



Encoder-Decoder Models (Seq2Seq Models)

- These models **map one sequence into another sequence**.
- The **encoder** reads the input sequence and creates a representation.
- The **decoder** uses that representation to generate an output sequence.
- **Very popular for transformation tasks**
- **Machine Translation**
 - Input: English sentence
 - Output: French/Urdu/Chinese sentence
- **Speech Recognition**
 - Input: audio features
 - Output: text
- **Summarization**
- **Paraphrasing**
- **Question generation**
- **Examples**
 - T5
 - BART
 - MarianMT
 - Whisper (speech-to-text has encoder-decoder design)
- These models are excellent when the job is to **convert one thing to another**.



Large
Language
Models

Conditional Generation of Text: The Intuition

Big idea

- **Big Idea:** Many tasks in NLP can be turned into “predict the next word” tasks
- This is one of the most important insights behind modern LLMs.



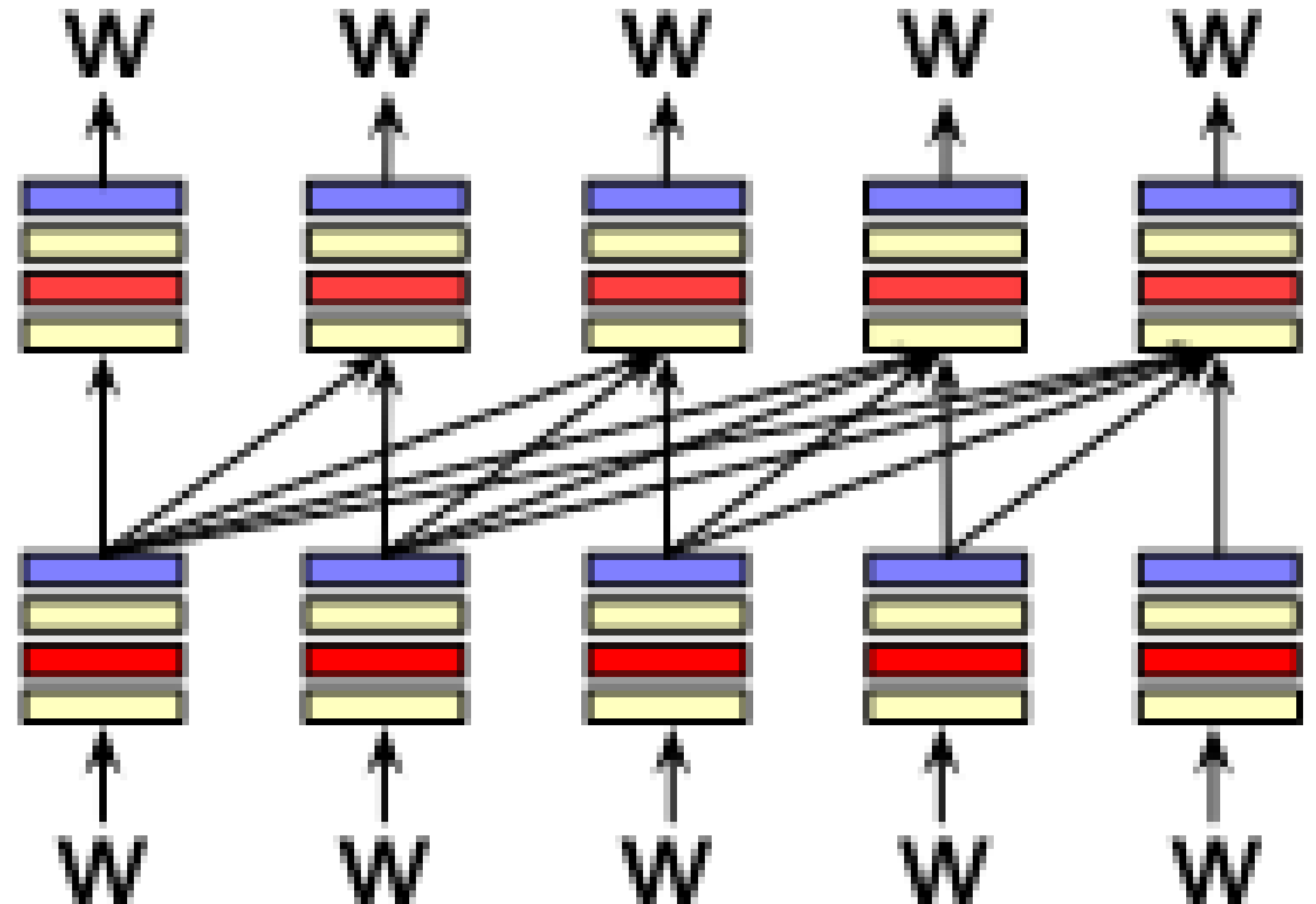
Big idea

- **1. Why is this a big idea?**
 - Because if you can convert many different tasks into a **word-prediction problem**, then:
 - You can train *one single model* (an LLM) to do many tasks
 - Using just one training objective: **predict the next word (or masked word)**
 - No need to build a separate model for every task
 - You don't need large labeled datasets for every task
 - LLMs learn a lot simply by **predicting words**.
- **2. Examples: How many NLP tasks become “predict the next word”?**
- **1. Question Answering**
 - Prompt:
 - Q: Who wrote Pride and Prejudice?
 - A: The model just predicts the next word:
 - A: Jane
 - A: Jane Austen
 - It *looks like* QA, but technically the model is just generating the next word(s).
- **2. Translation**
 - Translate English to Urdu:
 - "I love books" →
 - The model predicts:
 - میں کتابیں پسند کرتا ہوں
 - It *looks like translation*, but computationally it's next-word generation.
- **3. Summarization**
 - Summarize:
 - "The movie was long but well-directed..."
 - Summary:
 - The model predicts:
 - A well-directed but lengthy movie.
- **4. Information Extraction (NER, slots, etc.)**
 - Extract city:
 - "John traveled from Lahore to Karachi."
 - City:
 - Model predicts:
 - Karachi
 - Again, just next-word prediction.
- **5. Sentiment Analysis**
 - Is the sentiment positive or negative?
 - "The food was terrible."
 - Answer:
 - Model predicts:
 - negative
 - No classifier needed — just word prediction.
- **3. Why does this work?**
 - Because during pretraining the model learns:
 - grammar
 - facts
 - relationships
 - reasoning patterns
 - how questions and answers look
 - how translation pairs look
 - how summaries look
 - So when we prompt it, predicting the next token **looks like the task** we want it to do.



This lecture: decoder-only models

- Also called:
- Causal LLMs
- Autoregressive LLMs
- Left-to-right LLMs
- Predict words left to right



Conditional Generation:

- **Conditional generation** means:
 - The model generates text, but *conditioned* (influenced) by some input text.
 - That input text is usually called a **prompt**.
 - **How Conditional Generation Works (Step-by-Step)**
 - **1. You give the LLM a prompt**
 - Example: So long and thanks for
 - This prompt becomes the **context** for generating the next words.
 - **2. The model generates the output one token at a time**
 - It does **not** generate a whole sentence at once.
 - Instead, it repeats this cycle:
 - Look at the **previous text** (prompt + generated tokens)
 - Compute the **probability distribution** for the next token
 - **Sample** one token from that distribution
 - Add the token to the context
 - Repeat
 - **The Core Probability Formula**
 - The model tries to compute: $P(w_i | w_{<i})$
 - Where:
 - w_i = the next token
 - $w_{<i}$ = all previous tokens (the context)
 - **3. How the Model Generates Text**
 - At each step:
 - **Step A: Compute probabilities**
 - For example, it may compute:

Token	Probability
all	0.44
the	0.33
your	0.15
that	0.08
- These probabilities depend on the **entire context**.
- **Step B: Sample the next token**
- "Sampling" means:
 - The model picks a token according to the probabilities
 - Higher-probability tokens are more likely
 - Some randomness is allowed for creativity
- Suppose it chooses: all: 0.44
- **Step C: Add the token to the context**
- The prompt becomes:
 - So long and thanks for all
- And now it repeats:
 - Look at the updated context
 - Predict $P(w_{i+1} | \text{new context})$
- Sample the next token
- Add it
- Continue...

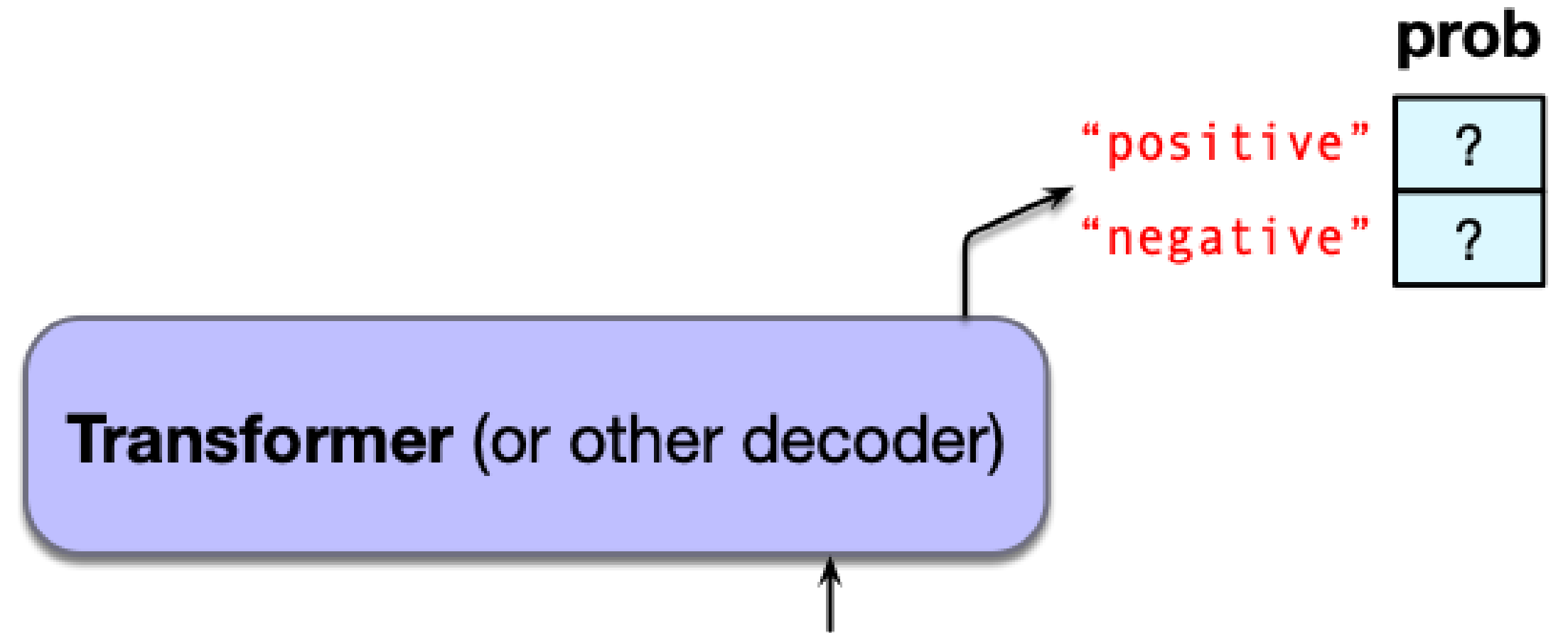


NLP Tasks can be Conditional Generation

- Many NLP tasks (including sentiment analysis) can be reframed as **conditional text generation**, where the model simply predicts the **next word** given a carefully designed **prompt**.
- The **prompt defines the task**.
Example prompt:
- To turn it into a **sentiment task**, you do NOT ask the model:
 - “I like Jackie Chan” → next word?
- Instead you ask:
 - The sentiment of the sentence "I like Jackie Chan" is:
- In this context, the model predicts the **most likely next word** that fits the pattern usually: **positive**
- **Why It Works**
- During pretraining, the model sees thousands of patterns like:
“*The sentiment is positive/negative.*”
- Words like “**like**” appear mostly in *positive contexts*, so the continuation
“... *is positive*”
becomes the highest-probability next token.
- Without the prompt,
 - I like Jackie Chan →
- the next word would be **movie, because, he**, etc.
- **But the prompt switches the task from “continue the sentence” to “classify sentiment.”**
- **Key Insight**
- **LLMs don’t run a classifier they just continue text, and the prompt makes the continuation equal to the answer.**



Sentiment via conditional generation



The sentiment of the sentence "I like Jackie Chan" is:

Which word has a higher probability?

$P(\text{positive} | \text{The sentiment of the sentence 'I like Jackie Chan' is:})$

$P(\text{negative} | \text{The sentiment of the sentence 'I like Jackie Chan' is:})$

Framing Lots Of Tasks As Conditional Generation

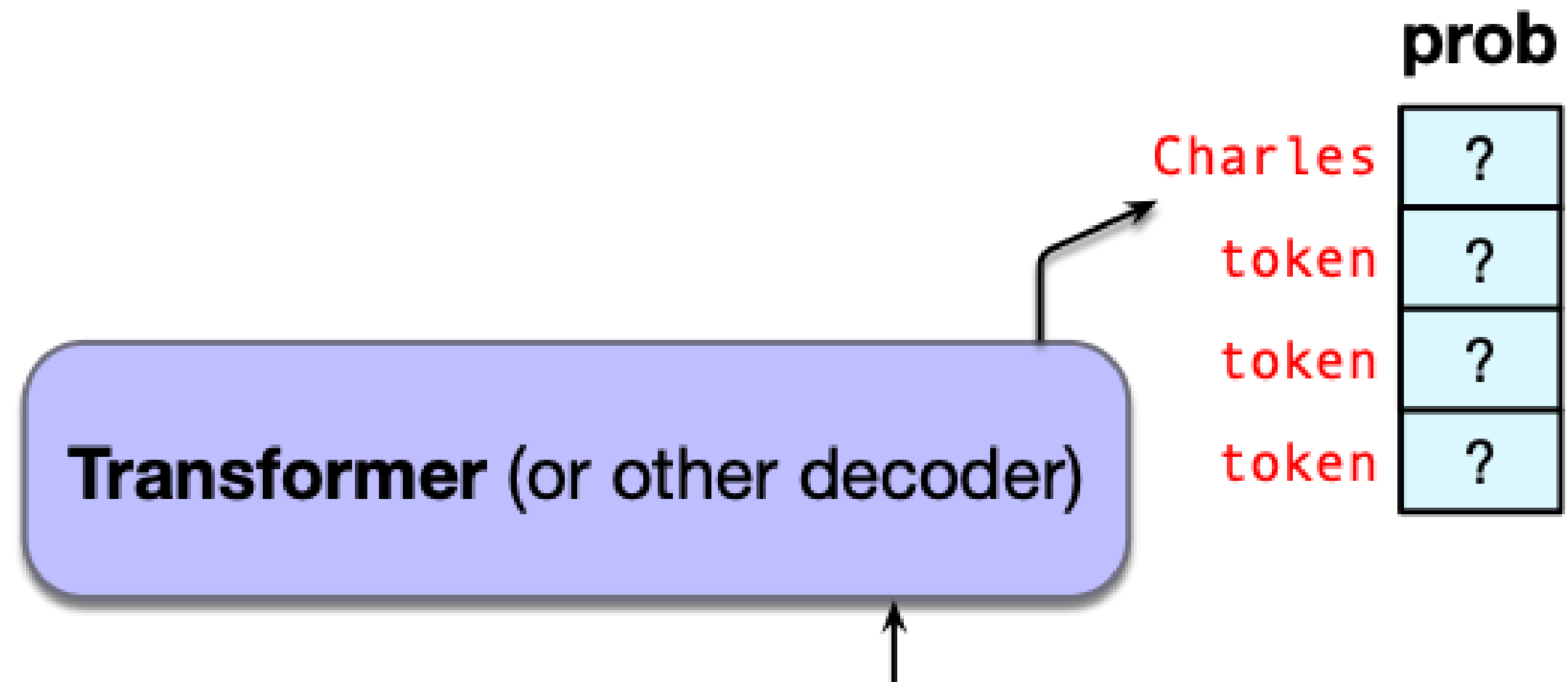
- QA: “Who wrote The Origin of Species”

- We give the language model this string:

Q: Who wrote the book ‘ ‘The Origin of Species’? A:

- And see what word it thinks comes next:

$P(w|Q: \text{Who wrote the book ‘ ‘The Origin of Species’? A:})$



Q: Who wrote the book 'The Origin of Species' A:

Now we iterate:

$P(w|Q: \text{Who wrote the book 'The Origin of Species' } A: \text{Charles})$

Large
Language
Models

Prompting

Prompt

- **Prompt:** a text string that a user issues to a language model to get the model to do something useful by conditional generation
- **Prompt engineering:** the process of finding effective prompts for a task.



Prompts

- A question:
 - What is a transformer network?
- Perhaps structured:
 - Q: What is a transformer network? A:
- Or an instruction:
 - Translate the following sentence into Hindi: 'Chop the garlic finely'.



Prompts can be very structured

A *structured prompt* is a prompt that follows a very clear, consistent **format**, **Why this works so well:** often resembling a form, template, or dialogue.

- LLMs respond better when the prompt has:
 - clear roles (e.g., “Human:” and “Assistant:”)
 - clear instructions
 - clearly defined answer formats
 - explicit choices
- Because LLMs learn patterns from massive amounts of text, using a **familiar structure** helps the model understand what you want and produce a more reliable answer.
- **Example**
- Human: Do you think that “input” has negative or positive sentiment?
- Choices:
 - (P) Positive
 - (N) Negative
- Assistant: I believe the best answer is: (
 - “**Human:**” tells the model this is the human’s message.
 - **The question is clear:** you are asking about sentiment classification.
 - **Choices are explicitly listed** (P or N).
 - “**Assistant:**” signals that the model should produce the answer.
 - The model now knows:
 - It must produce **only (P) or (N)**.
 - It must answer in a format it recognizes.
 - The structure **reduces ambiguity**, so the model doesn’t produce long sentences like:
 - “This is a positive statement because...”
 - Instead, it is more likely to produce:
 - (P)
 - This is why structured prompts are powerful.

Prompts can have demonstrations (= examples)

A *demonstration* (also called an example) is when you show the model **how you want it to answer** by giving it one or more solved examples *before* the actual question.

- This technique is called **few-shot prompting**.
- **Why this helps:**
- LLMs are fundamentally **pattern learners**.
If you show them a pattern, they mimic/copy it.
- Examples “teach” the model:
 - the style of the question
 - how the answer should look
 - what level of detail is required
 - how multiple-choice answers are formatted
 - how tasks are solved
- This drastically improves accuracy.
- **Example**
- The following are multiple choice questions about high school

computer science.

- **Demonstrations 1**

Let $x = 1$. What is $x \ll 3$ in Python 3?

(A) 1 (B) 3 (C) 8 (D) 16

Answer: C

- **Demonstrations 2**

Which is the largest asymptotically?

(A) $O(1)$ (B) $O(n)$ (C) $O(n^2)$ (D) $O(\log n)$

Answer: C

- **Ask Question Now**

What is the output of the statement "a" + "ab" in Python 3?

(A) Error (B) aab (C) ab (D) a ab

Answer:



Prompts can have demonstrations (= examples)

- **What the model learns from these demonstrations** question:
 - By seeing the first two examples, the model learns:
 - **Every question is a multiple-choice question**
 - **Each question is followed by options labeled (A), (B), (C), (D)**
 - **Each question ends with “Answer: X”**
 - Not with an explanation, not with paragraphs
 - **The correct answer letter is placed directly after “Answer:”**
 - **The task is computer science related**, so it should use its CS knowledge
 - So when the model reaches the final unsolved
- What is the output of the statement "a" + "ab" in Python 3?
 - (A) Error (B) aab (C) ab (D) a ab
 - Answer:
 - The model already knows **exactly how to answer**, because the examples created a pattern.
 - The model will likely output:
 - Answer: B
 - because "a" + "ab" -> "aab" in Python.



Why Demonstrations Improve Model Performance

1. Why LLMs Need This: The Core Reason

- LLMs are not rule-based systems.
They don't "know" tasks unless the prompt clearly tells them:
- What the task is
- How the task should be answered
- What format to follow
- What style to mimic
- Structured prompts + demonstrations essentially "program" the model using language.

2. When Should You Use Demonstrations?

- Use them when you want the model to be:
 - more accurate
 - more consistent
 - more predictable
 - less verbose
 - more aligned with a specific style
 - Especially useful for:
 - multiple-choice questions
 - mathematical steps
 - classification tasks
 - reasoning patterns
 - code generation styles
 - extraction tasks
 - formal or template-based answers

Prompts are a learning signal

Prompts Are a Learning Signal But NOT the Same as Training

- When you give an LLM a prompt, the model **responds as if it is learning**, but nothing inside the model's parameters actually changes.
- This is why we call prompting a **learning signal**, but **not real weight-updating learning**.
- Let's break this down step by step.

1. Pretraining = REAL Learning (Weights Change)

- During pretraining:
 - The model predicts billions of next tokens.
 - After each prediction, the model computes loss.
 - Gradients are computed.
- **Weights (parameters) are updated via gradient descent.**
- This means the model **actually learns**, in the true machine learning sense.
- Knowledge is stored permanently in the weights.
This is why the model “remembers” grammar, facts, styles, reasoning patterns.

2. Prompting = No Weight Updates (Just Context)

- When you give a prompt like:
Q: What is 2 + 2?
A: 4
Q: What is 3 + 5?
A:
 - The model **does NOT**:
 - change its weights
 - update gradients
 - store new information permanently
 - Instead, the prompt:
 - changes the **activation patterns** inside the network
 - shapes how attention distributes over the tokens
 - tells the model “which ability” to activate
 - makes the next token probabilities fit the pattern shown
 - But once the prompt disappears → **the model forgets everything.**
 - This is **NOT** real learning.
 - It is **context-dependent behavior**.



What Is In-Context Learning (ICL)?

- In-Context Learning = the model appears to learn a task from examples in the prompt, even though NO parameters are changed.
- ICL works because:
 - The model's attention layers interpret the examples
 - Activations form **temporary representations**
 - The model generalizes inside that one context window
- When you remove the prompt, the ability disappears
- So the “learning” is:
 - Temporary
 - context-based
 - pattern-driven
 - activation-level
 - NOT stored in weights
 - NOT permanent
- 5. Why Does It Feel Like Real Learning?
 - Because the model has so much pretrained knowledge that when you give it:
 - a pattern
 - an example format
 - an instruction
 - demonstrations
 - ...its probability distribution reshapes itself to **simulate** learning.
 - It is really doing:
 - “Given these examples, what is the next most likely answer?”
 - Instead of:
 - “I have now permanently learned the concept of sentiment classification.”



LLMs Usually Have a System Prompt

What Is a System Prompt

- Large Language Models typically use **three roles** in a conversation:
 - **System**
 - **User**
 - **Assistant**
- The **system prompt** is a special, hidden instruction that tells the model **how it should behave** for the entire conversation.
- You usually don't see it, but it is **always added before your message**.
- **Example:**
- **<system> You are a helpful and knowledgeable assistant. Answer concisely and correctly.**
- This tells the model:
 - what identity it should take
 - what tone to use
 - how detailed its answers should be
 - what rules it should follow

- When you ask a question like:
 - **What is the capital of France?**
- The model does **NOT** see only this.
Instead, internally, the conversation becomes:
 - <system> You are a helpful and knowledgeable assistant. Answer concisely and correctly.**
 - <user> What is the capital of France?**
 - <assistant> ...**
- This means the model interprets your question **in the context of the system prompt**.
- Because of the system prompt, it answers: **Paris**
- instead of a long story, or instead of answering in a rude or random way.

2. The System Prompt Is Automatically Combined With Your Prompt



Why Do LLMs Need a System Prompt?

It sets the behavioral baseline

- Without a system prompt, the model has no instruction on how to behave. The system prompt enforces:
 - helpfulness
 - politeness
 - conciseness
 - correctness
 - tone/style
- **It acts as the “rules of the conversation”**
- The user prompt gives the *task*,

the system prompt gives the *personality and rules*.

It ensures consistency

- Every conversation starts in the same “mindset,” regardless of what the user asks.

Why You Don’t See the System Prompt

- Because it is applied automatically by the interface (*ChatGPT, Copilot, Claude, etc.*) to ensure consistent behavior.
- Unless you are using APIs or special modes, you won’t see it.



System prompts can be long; 1700 words for Claude Opus4

- **Why System Prompts Can Be Very Long (e.g., 1700 words for Claude Opus)**
- Modern LLMs (Claude, ChatGPT, Copilot, Llama) often have a **hidden system prompt** that sets their behavior.

This system prompt can be extremely long sometimes **hundreds or thousands of words**.

- Why?
 - Because the system prompt acts like the model's **operating system**, giving it high-level rules for:
 - personality
 - tone
 - safety
 - answer style
 - level of detail
 - what topics are allowed or forbidden
 - how to format output

System Prompts Can Be Long; 1700 Words For Claude Opus4

1. “Claude should give concise responses to very simple questions, but thorough responses to complex and open-ended questions.”

- This rule controls **adaptiveness**.
- If the user asks: “What is 2+2?” → Claude should answer briefly (“4”).
- If the user asks: “Explain quantum entanglement” → Claude should give a long, deep, structured explanation.
- This prevents:
 - unnecessary verbosity
 - overly short answers to complex topics

2. “Claude is able to explain difficult concepts clearly. It can also illustrate explanations with examples, thought experiments, or metaphors.”

- This defines **teaching style**.
- Claude is instructed to:
 - simplify hard ideas
 - use analogies
 - use metaphors
 - use examples
 - make explanations easy to understand
- This is why Claude often explains things using:
 - simple analogies
 - “imagine you are...” scenarios
 - relatable metaphors
- The system prompt shapes that behavior.



System Prompts Can Be Long; 1700 Words For Claude Opus4

3. “Claude does not provide information that could be used to make chemical or biological or nuclear weapons.”

- This is a **safety rule**.
- The system prompt must prevent Claude from generating:
 - dangerous instructions
 - weapon-making information
 - harmful scientific procedures
 - anything that could enable violence
- Long system prompts often include **many detailed safety guidelines**, because safety is complex and must cover many areas.

4. “For casual, emotional, empathetic, or advice-driven conversations, Claude keeps its tone natural, warm, and empathetic.”

- This controls the model’s **emotional tone**.
- Claude is instructed to:
 - sound friendly
 - sound caring
 - sound supportive
 - use natural, human-like emotional warmth
 - avoid being robotic or cold in sensitive situations
- This is important for:
 - mental health support
 - emotional conversations
 - motivational or personal advice
 - customer support style dialogue



System Prompts Can Be Long; 1700 Words For Claude Opus4

- 5. “Claude cares about people’s well-being and avoids encouraging or facilitating self-destructive behavior.”
- Another safety requirement.
- Claude must:
 - avoid harmful encouragement
 - not enable self-harm
 - avoid dangerous advice
 - show empathy and care
- This ensures the model behaves responsibly even in sensitive contexts.
- 6. “If Claude provides bullet points, they should be in markdown, and each bullet point should be 1–2 sentences long unless requested otherwise.”
- This is a **formatting instruction**.
- Claude's system prompt defines:
 - when to use bullets
 - how long bullet points should be
 - that bullets should use Markdown (- or *)
 - how structured the answer should look
- This ensures consistent, well-formatted responses.



Why So Many Rules?

- Because the goal of the system prompt is to define **the assistant's personality and behavior** very precisely.
- A long system prompt helps the model behave:
 - consistently
 - safely
 - with appropriate tone
 - with good formatting
 - with helpfulness
 - with clarity
- So even though the user only sees:
 - You are Claude. How can I help you?
- Behind the scenes, Claude is operating under a long, detailed specification like a **constitution**.
- **Key Takeaway**
- **Long system prompts define how an LLM behaves across all conversations. They are like the assistant's internal instruction manual.**
- Claude's 1700-word system prompt includes rules about:
 - conciseness vs depth
 - clarity and teaching style
 - safety (no harmful info)
 - emotional warmth
 - well-being
 - formatting guidelines
- These rules ensure the model behaves predictably and responsibly.



Large
Language
Models

Sampling for LLM Generation

Recap Neural Network Logits

Logits are the raw scores that a neural network produces *before* converting them into probabilities.

- They come directly from the final **linear layer** of the model.
 - They are **not probabilities**
 - They can be **positive or negative**
 - They can be **any real number** (unbounded)
- The softmax function later turns these logits into a probability distribution.

Softmax Turns Logits into Probabilities

- Softmax converts the logits into values that:
 - are between 0 and 1
 - sum to 1

When are numbers called *logits*?

• **A vector is called *logits* ONLY when it:**

- has length = number of classes/tokens
- represents class scores
- is intended to go into softmax or cross-entropy

Can an intermediate layer produce logits?

- **YES**, if you intentionally design it to output class scores.

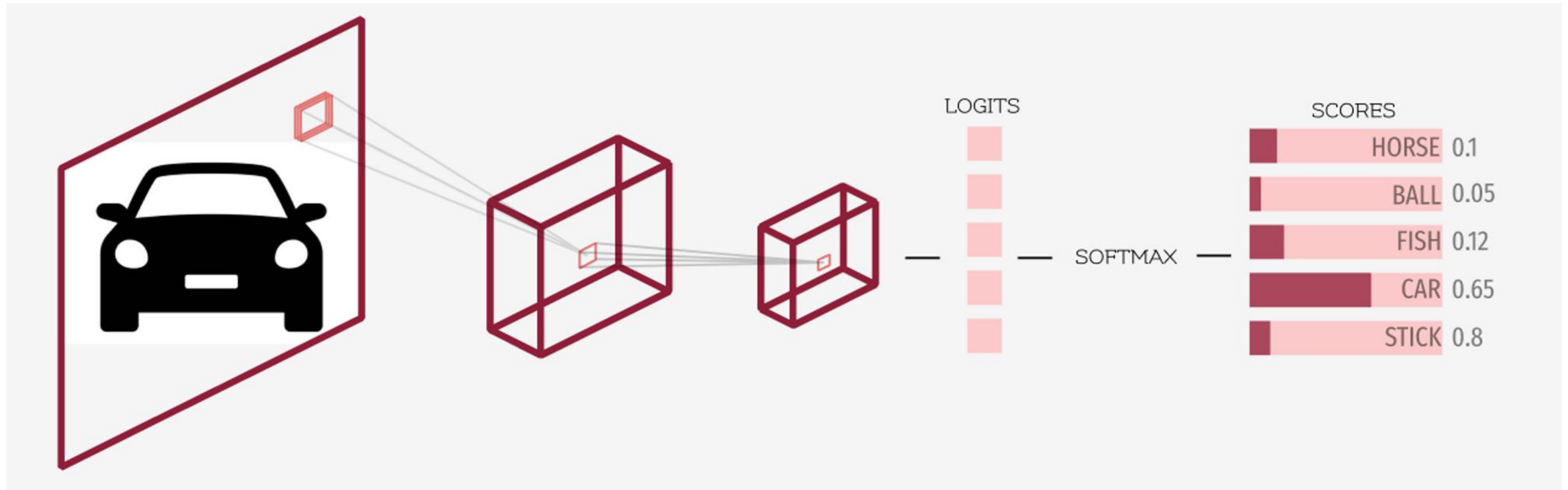
Do typical networks do this?

- **NO** typically only the final layer outputs logits.

Does every linear layer produce logits?

- **NO** only linear layers that output class-score vectors do.

Recap Neural Network Logits



Example: Linear Transformation in the Final Layer

- **Softmax:** $\sigma(\vec{Z})_i = \frac{e^{Z_i}}{\sum_{j=1}^K e^{Z_j}}$
- **Softmax in 3 easy steps**
- If your final vector has **three numbers**, for example:
 - $u = [2, 1, 0]$
- Then softmax works like this:
- **Step 1: Take exponential of each number**
 - e^2
 - e^1
 - e^0
- **Step 2: Add all exponentials**
 - $S = e^2 + e^1 + e^0$
- **Step 3: For each number, divide its exponential by the sum**
- For example, the first probability:
 - $p_1 = \frac{e^2}{S}$
 - Similarly:
 - $p_2 = \frac{e^1}{S}, p_3 = \frac{e^0}{S}$
- **Quick Summary**
 - **Softmax = exponential of a number ÷ sum of exponentials of all numbers.**

Example: Linear Transformation in the Final Layer

- Suppose we want to classify something into **3 classes**:
 - Class A
 - Class B
 - Class C
- So the final layer must output **3 numbers = 3 logits**.
- Let's say the last hidden vector (from the network) is:

- $h = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$
- So the hidden size is **2**.

- Now, we apply a **linear layer**:

- $u = Wh + b$

- Where:

- **Weight matrix $W(3 \times 2)$:**

- $W = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ -1 & 1 \end{bmatrix}$

- **Bias vector $b(3 \times 1)$:**

- $b = \begin{bmatrix} 0 \\ 1 \\ -2 \end{bmatrix}$

- **Step 1: Multiply W and h**

- $Wh = \begin{bmatrix} 1 \cdot 2 + 0 \cdot (-1) \\ 0 \cdot 2 + 2 \cdot (-1) \\ -1 \cdot 2 + 1 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \\ -3 \end{bmatrix}$

- **Step 2: Add bias b**

- $u = Wh + b = \begin{bmatrix} 2 \\ -2 \\ -3 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ -5 \end{bmatrix}$

- **These 3 numbers are the *logits*!**

- $logits = \begin{bmatrix} 2 \\ -1 \\ -5 \end{bmatrix}$

- These are the raw scores for the 3 classes before softmax.



Example: Linear Transformation in the Final Layer

- **Step 3 (Optional): Apply Softmax to get probabilities**

- $$\text{softmax}(u_i) = \frac{e^{u_i}}{\sum_j e^{u_j}}$$

- Compute exponentials:

- $e^2 \approx 7.39$

- $e^{-1} \approx 0.37$

- $e^{-5} \approx 0.0067$

- Sum:

- $7.39 + 0.37 + 0.0067 \approx 7.7667$

- Probabilities:

- Class A: $7.39/7.7667 \approx 0.95$

- Class B: $0.37/7.7667 \approx 0.048$

- Class C: $0.0067/7.7667 \approx 0.00086$

- **The model predicts Class A.**

Final Takeaway

- A **linear transformation** in the final layer:

- $u = Wh + b$

- produces **logits** raw class scores.

- Softmax then turns logits into probabilities.

- This is exactly what happens in:

- image classifiers

- sentiment classifiers

- LLM next-token prediction

- Every time!



Where does token probability come from?

- For every possible token in the vocabulary, it produces a real-valued score called a *logit*.

1. The LLM processes the input and produces hidden representations

- When you give a prompt like:
 - The capital of France is
- the Transformer layers compute internal vectors representing the context (attention, feed-forward networks, etc.).
- The last hidden layer produces a vector representation for the final token.

2. The final linear layer converts hidden states → logits

- The last step is a **linear transformation**:
- $u = W \cdot h + b$

- Where:

- h = final hidden state for the last token
- W = output weight matrix of shape $[|V| \times d]$
- $|V|$ = vocabulary size (e.g., 32k tokens)
- u = vector of **logits**, one score per token

- So the model produces something like:

- $\text{logit}(\text{"Paris"}) = 11.2$
- $\text{logit}(\text{"London"}) = 9.1$
- $\text{logit}(\text{"Berlin"}) = 8.3$
- $\text{logit}(\text{"dog"}) = -5.6$
- $\text{logit}(\text{"banana"}) = -7.8$
- ... (one for every token)

- These numbers are **not yet probabilities**.
- They are unbounded scores (can be positive or negative).



Where does token probability come from?

3. The logit vector is passed through a softmax to get probabilities

- Softmax converts raw scores into a **valid probability distribution**.

$$y_i = \frac{e^{u_i}}{\sum_j e^{u_j}}$$

- This ensures:
- all probabilities are between 0 and 1
- all probabilities sum to 1
- So after softmax, we get:
 - $P(\text{"Paris"}) = 0.85$
 - $P(\text{"London"}) = 0.10$
 - $P(\text{"Berlin"}) = 0.04$
 - $P(\text{"dog"}) = 0.00001$
 - ...
- The highest-probability token is **"Paris"**, so the model predicts it as the next token.

4. Why "Paris" gets a higher logit

- Because during pretraining the model saw countless examples like:

- "The capital of France is Paris"
- "France's capital city, Paris..."
- "Paris is the capital and largest city of France"

- So the internal networks learn patterns that give **Paris** a higher score for this context.

5. The Probability Pipeline

- Input text → transformed through layers
- Final hidden state h → fed into output matrix
- Output matrix produces **logits** $u \in \mathbb{R}^{|V|}$
- Softmax converts logits → probabilities
- Model samples or selects the next token

One-Sentence Explanation

- **Token probabilities come from transforming the hidden state into logits for every vocabulary token, then applying softmax to convert those logits into a probability distribution.**



Where does token probability come from?

- $u \in \mathbb{R}^{|V|}$
- $y = \text{softmax}(u)$

1. What does $u \in \mathbb{R}^{|V|}$ mean?

- u is a **vector** (a list of numbers).
- It has **$|V|$ components**, meaning one number for **each token in the vocabulary**.
- If your vocabulary has:
- 50,000 tokens $\rightarrow u$ has 50,000 numbers
- 32,000 tokens $\rightarrow u$ has 32,000 numbers
- Example vocabulary size:
- $|V| = 5$
- Then: $u = [u_1, u_2, u_3, u_4, u_5]$
- These numbers are called **logits** (raw, unnormalized scores).

2. Shape $[1 \times |V|]$

- 1 row
- $|V|$ columns (one score per token)
- Example: If $|V| = 5$:
- $[2.4 \quad -0.1 \quad 3.7 \quad 0.9 \quad -4.2]$
- **3. Softmax converts logits into probabilities**
- Softmax takes the logit vector u and produces a **probability distribution** y .

- The formula for each element y_i is: $y_i = \frac{e^{u_i}}{\sum_{j=1}^{|V|} e^{u_j}}$

- Take the exponential of each logit $e^{u_1}, e^{u_2}, \dots, e^{u_{|V|}}$

- Add all exponentials

- $S = \sum_{j=1}^{|V|} e^{u_j}$

- Divide each exponential by the sum

- $y_i = \frac{e^{u_i}}{S}$



Where does token probability come from?

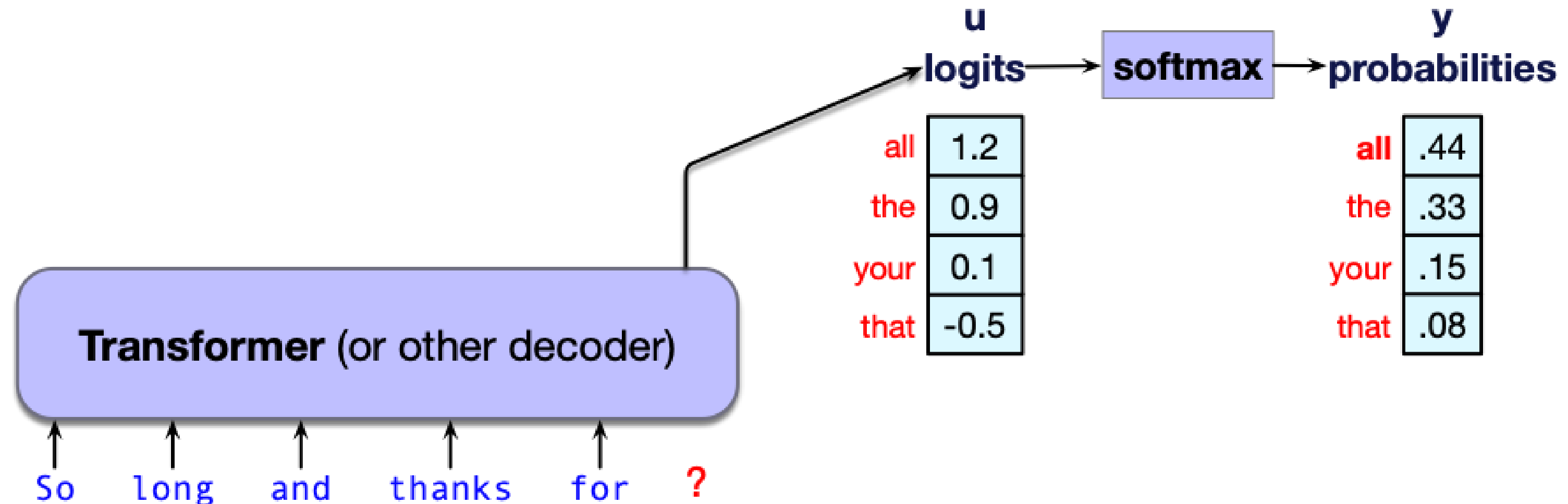
- **Small numerical example**
- Suppose vocabulary has 3 tokens:
- $|V| = 3$
- $u = [2, 1, 0]$
- Compute exponentials:
 - $e^2 = 7.389$
 - $e^1 = 2.718$
 - $e^0 = 1$
- Sum:
 - $S = 7.389 + 2.718 + 1 = 11.107$
- Now compute probabilities:
- $y_1 = 7.389/11.107 = 0.665$
- $y_2 = 2.718/11.107 = 0.245$
- $y_3 = 1/11.107 = 0.090$
- So:
- $y = [0.665, 0.245, 0.090]$
 - These are valid probabilities
 - They sum to 1
 - The model will likely choose the first token
- **1. Token → Embedding**
 - 300-4096 dimensions
- **2. Process through Transformer layers**
 - Still many dimensions (hidden size)
- **3. Final linear projection → Logits**
 - $|V|$ dimensions (one score per token)
- **4. Softmax → Probability distribution**
 - $|V|$ probabilities that sum to 1



Where does token probability come from?

- The internal networks for LLMs generate real-valued scores called **logits** for each token in the vocabulary.
- Score vector **u** of shape $[1 \times |V|]$ is turned into a probability by softmax

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$



Decoding

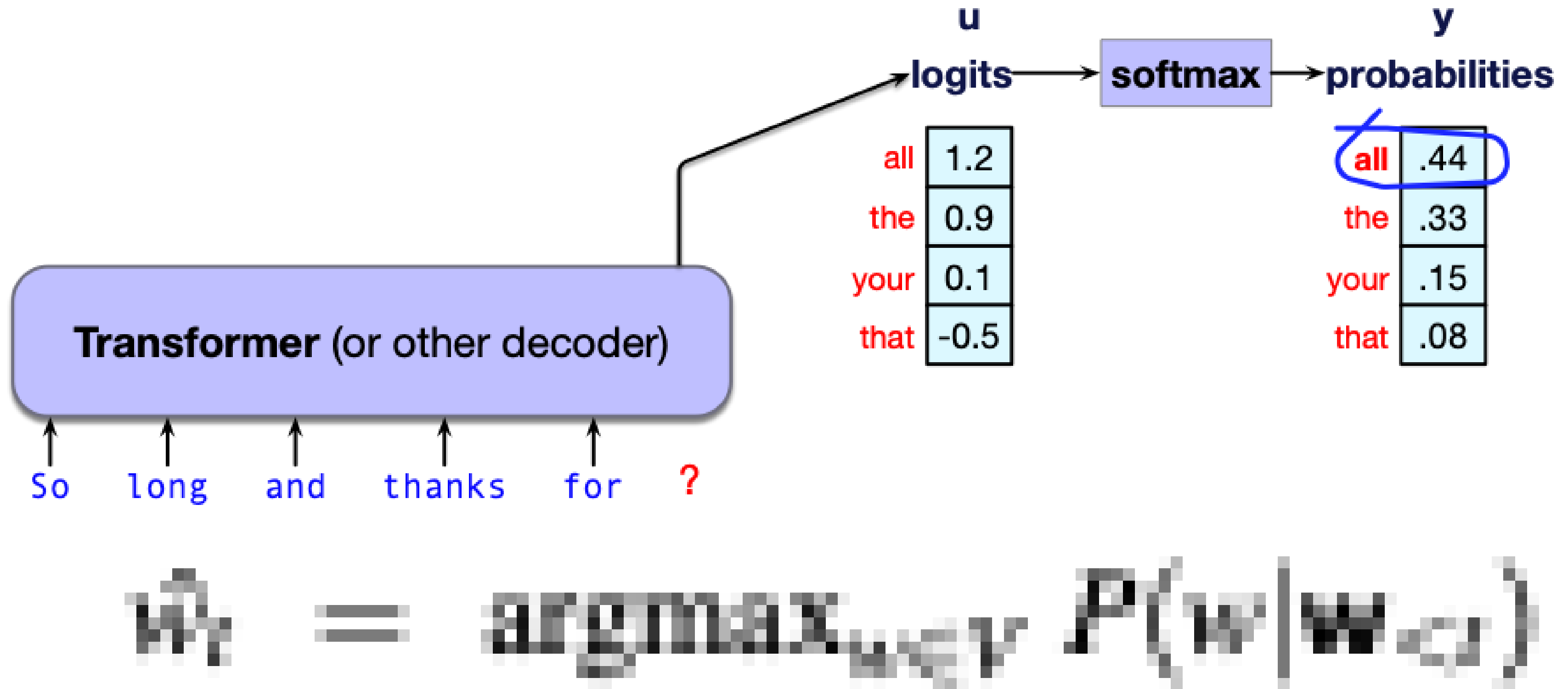
- After the model computes **probabilities for each next token** (using logits $\rightarrow$ softmax), it must **choose one token** to output.
- This choosing step is called **decoding**.
- Think of it like this:
- The model gives you a list of probabilities for all tokens
- You must pick one token to generate
- The strategy you use to pick the token = **decoding method**
- **Examples of decoding methods**
 - Greedy decoding
 - Sampling
 - Top-k sampling
 - Top-p (nucleus) sampling
 - Beam search
 - But for now, just remember:
 - **Decoding = choosing the next word according to the model's probabilities.**

Greedy Decoding

- A **greedy algorithm** always chooses the **best immediate option** at each step even if that choice may not lead to the best long-term result.
- In language models:
- **Greedy decoding = always pick the token with the highest probability.**
- It does NOT consider future possibilities.
- It may miss better sentences overall.
- **Mathematical Formula**
 - $\hat{w}_i = \operatorname{argmax}_{w \in V} P(w \mid w_{<i})$
 - $\hat{w}_i$ = the chosen next token
 - V = the vocabulary
- $P(w \mid w_{<i})$ = the probability of token **w** given all previous tokens
- **argmax** = choose the token **w** that has the **largest probability**
- So the model does:
 - Look at all possible next tokens
 - Compute probability for each
 - Pick the **highest-probability** one
 - Add it to the output
 - Repeat
- If two or more tokens have exactly the same highest probability, model will choose the first one in its internal vocabulary order.



Greedy decoding: choosing "all"



We don't use greedy decoding

Reason 1: Predictability → Repetition

- Because greedy decoding is predictable:
 - It often gets stuck in **loops**
 - It often repeats phrases
 - It produces boring patterns like:
“I think the best way to do this is to think about the best way to do this...”
- Why?
 - Because the highest-probability token is often a **common**, **safe**, and **generic** word.
 - And if that word is always chosen, the model keeps spiraling into repetitive patterns.

Reason 2: Deterministic Behavior

- Greedy decoding is **deterministic**.
- That means:
 - Same input → same output
 - Every time
 - No variation
 - No creativity
- This is why greedy decoding is **too predictable** for generating natural text like stories, answers, or conversation.

Reason 3: It Ignores Future Possibilities

- Greedy decoding only cares about **this immediate step**, not long-term sentence quality.

- Imagine writing a story where you always choose the next word based **ONLY** on the strongest short-term guess.
You would never explore interesting or creative wording because you always pick “safe” words.

- **People Prefer TEXT WITH VARIETY → so they use Sampling**
- To get more natural, human-like text, we introduce **randomness**.
- **Greedy decoding = always pick the highest-probability token**
 - → deterministic
 - → predictable
 - → generic
 - → repetitive
- **Sampling = pick according to probability distribution**
 - → diverse
 - → creative
 - → more natural

We don't use greedy decoding

Reason 1: Predictability → Repetition

- Because greedy decoding is predictable:
 - It often gets stuck in **loops**
 - It often repeats phrases
 - It produces boring patterns like:
“I think the best way to do this is to think about the best way to do this...”
- Why?
 - Because the highest-probability token is often a **common**, **safe**, and **generic** word.
 - And if that word is always chosen, the model keeps spiraling into repetitive patterns.
- Greedy decoding only cares about **this immediate step**, not long-term sentence quality.
 - Imagine writing a story where you always choose the next word based ONLY on the strongest short-term guess.
You would never explore interesting or creative wording because you always pick “safe” words.
- **People Prefer TEXT WITH VARIETY → so they use Sampling**
- To get more natural, human-like text, we introduce **randomness**.
- **Greedy decoding = always pick the highest-probability token**

Reason 2: Deterministic Behavior

- Greedy decoding is **deterministic**.
 - Same input → same output
 - Every time
 - No variation
 - No creativity
- This is why greedy decoding is **too predictable** for generating natural text like stories, answers, or conversation.
- → deterministic
- → predictable
- → generic
- → repetitive
- **Sampling = pick according to probability distribution**
 - → diverse
 - → creative
 - → more natural

Reason 3: It Ignores Future Possibilities



Random Sampling

- **Sampling = pick according to the probability distribution**
- It means:
- Tokens with **higher probability** are *more likely*
- Tokens with **lower probability** are *less likely*
- BUT **any token** with non-zero probability *can be chosen*
- Not just the highest one.
- **Example to Make It Crystal Clear**
- Suppose the model gives probabilities for next token:
- **Token , Probability**
- “the” , 0.60
- “a” , 0.25
- “it” , 0.10
- “this” 0.05

Greedy decoding:

- ALWAYS picks:
- “the”

Sampling:

- You **roll a weighted dice**:
- “the” has a 60% chance
- “a” has a 25% chance
- “it” has a 10% chance
- “this” has a 5% chance

So sampling might pick:

- “the” (most of the time)
- “a” (sometimes)
- “it” (rarely)
- “this” (very rarely)
- This adds **diversity** and **creativity**.

